



Namal University, Mianwali

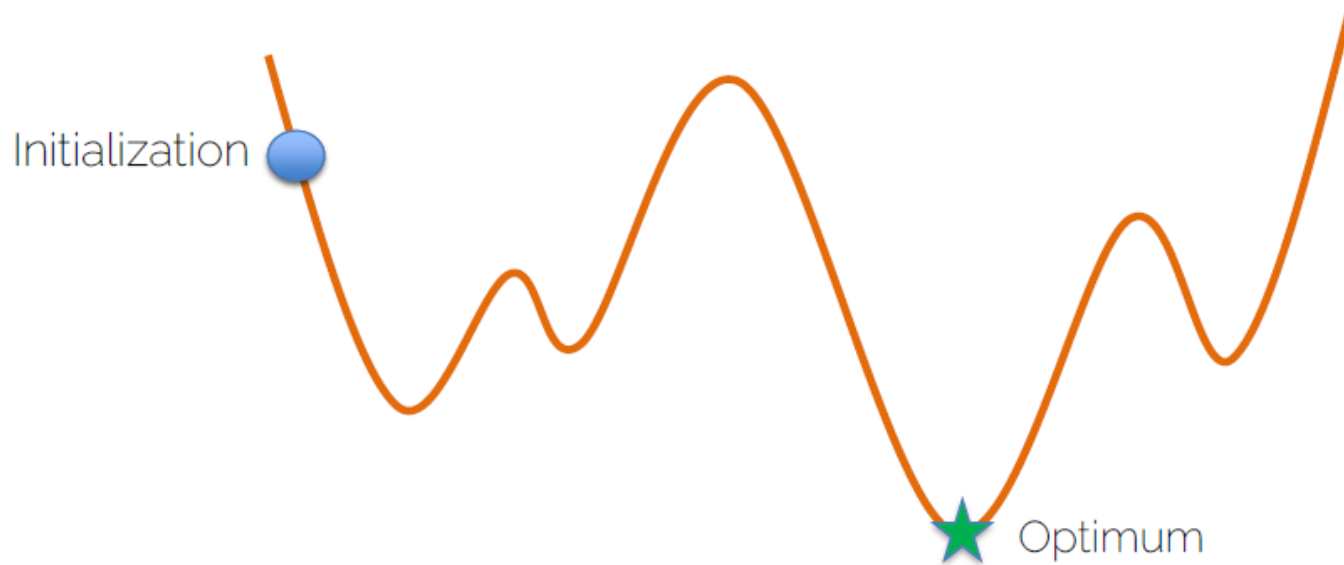
Department of Computer Science

Machine Learning

Optimizers

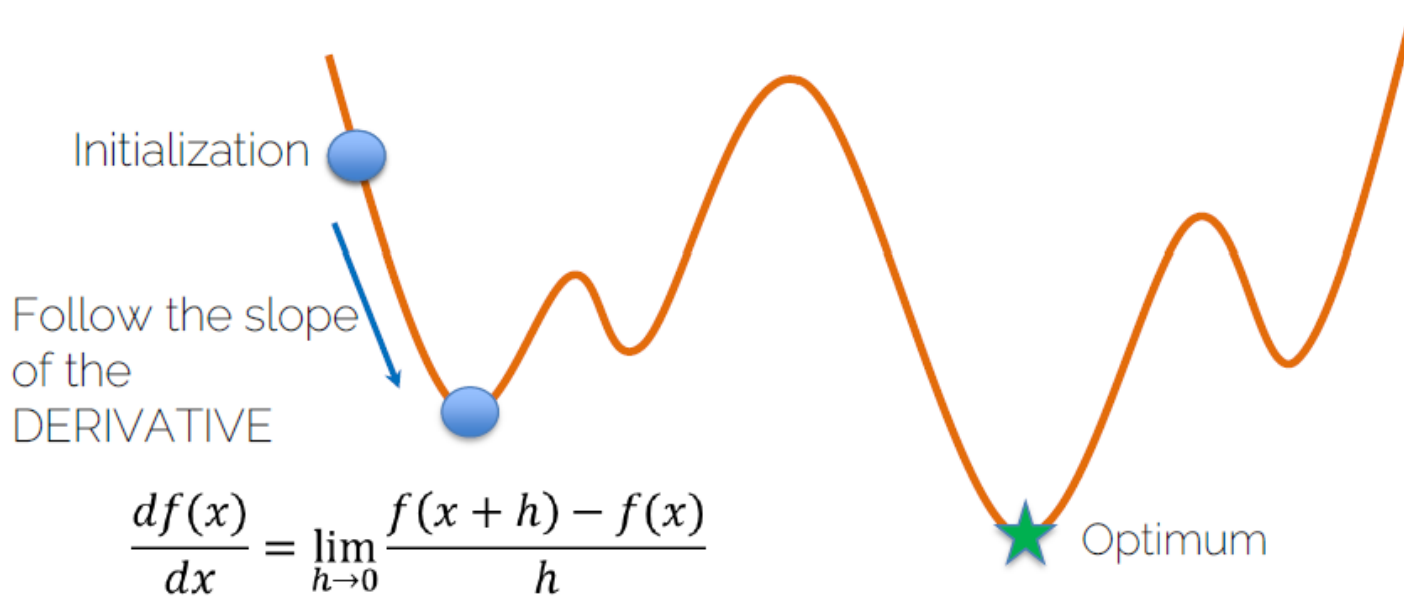
Gradient Descent

$$x^* = \arg \min f(x)$$

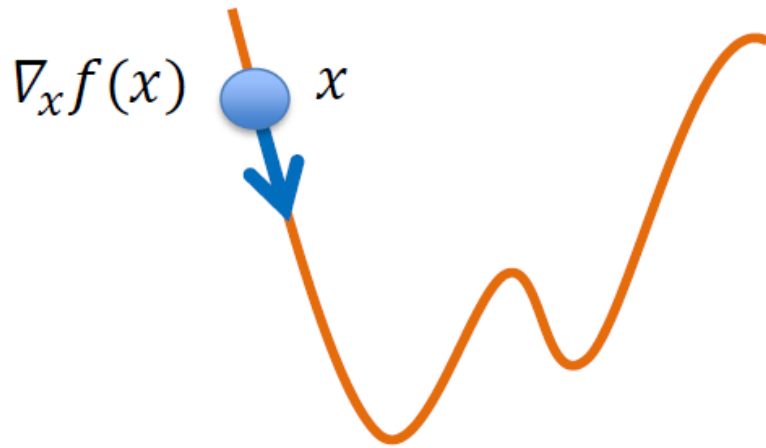


Gradient Descent

$$x^* = \arg \min f(x)$$



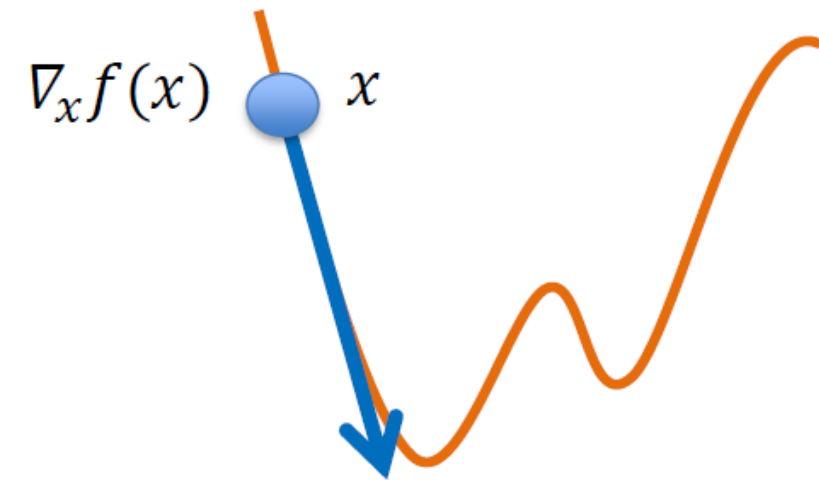
Gradient Descent



$$x' = x - \alpha \nabla_x f(x)$$

SMALL Learning rate

Gradient Descent



$$x' = x - \alpha \nabla_x f(x)$$

LARGE Learning rate

Gradient Descent

$$\mathbf{x}^* = \arg \min f(\mathbf{x})$$

Initialization

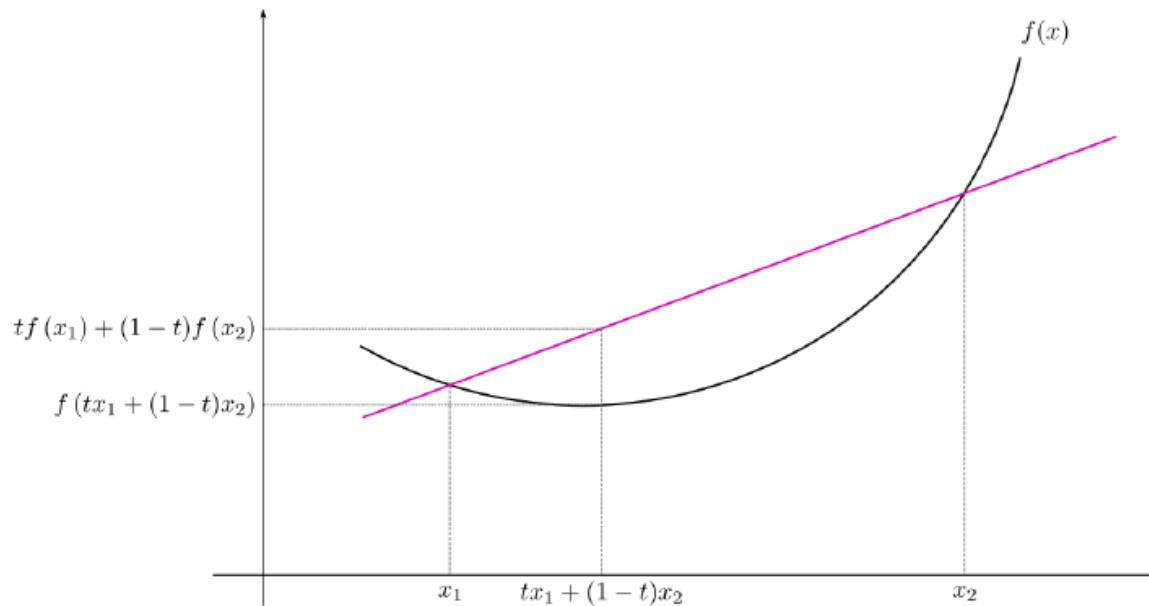
What is the gradient when we reach this point?

Optimum

Not guaranteed to reach the global optimum

Convergence of Gradient Descent

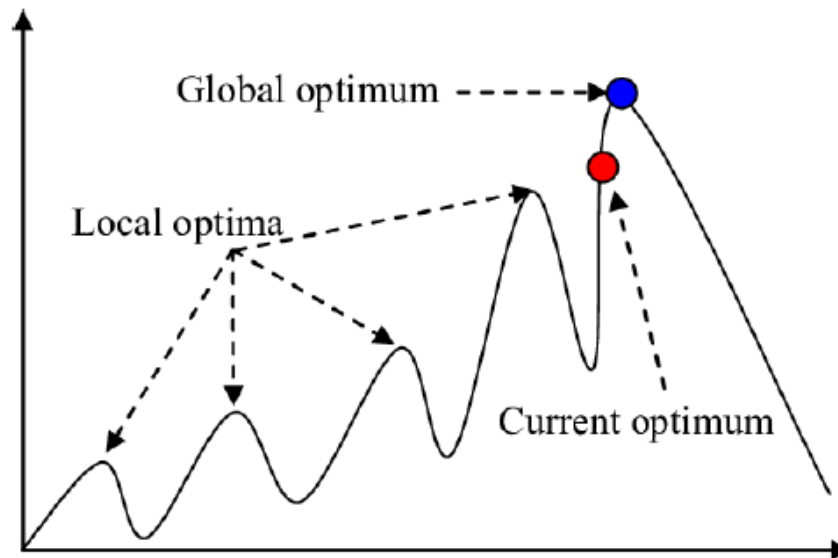
Convex function: all local minima are global minima



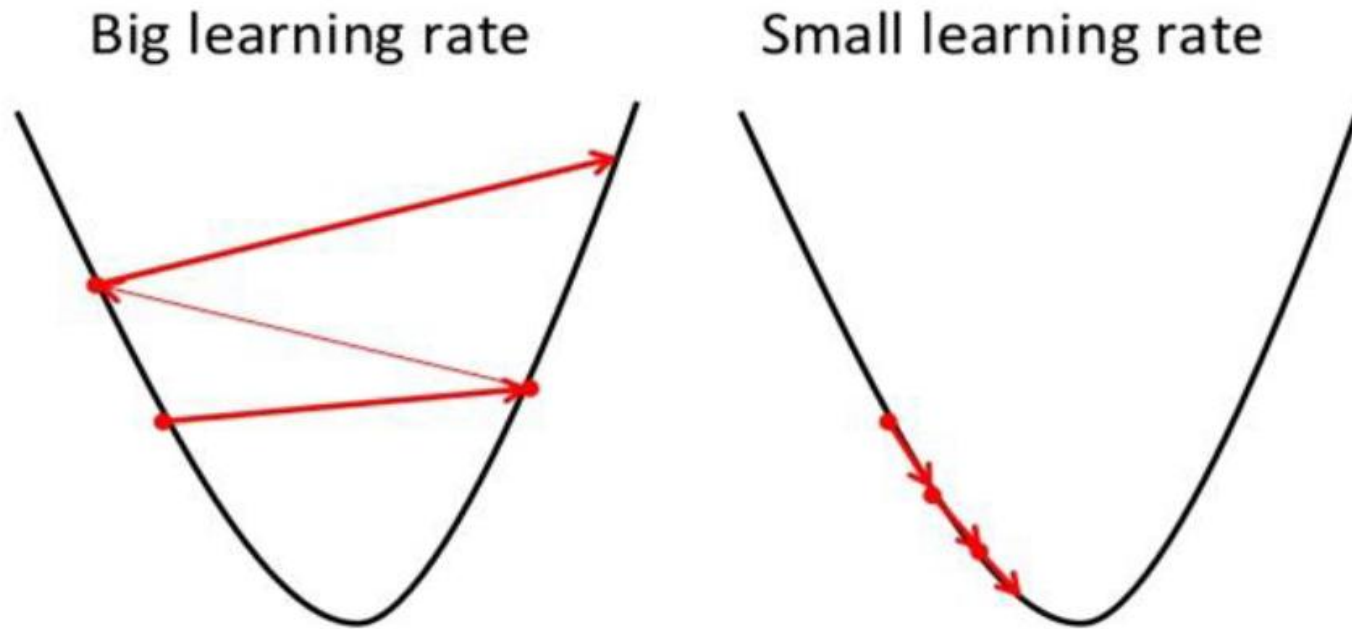
Convergence of Gradient Descent

Neural networks are non-convex

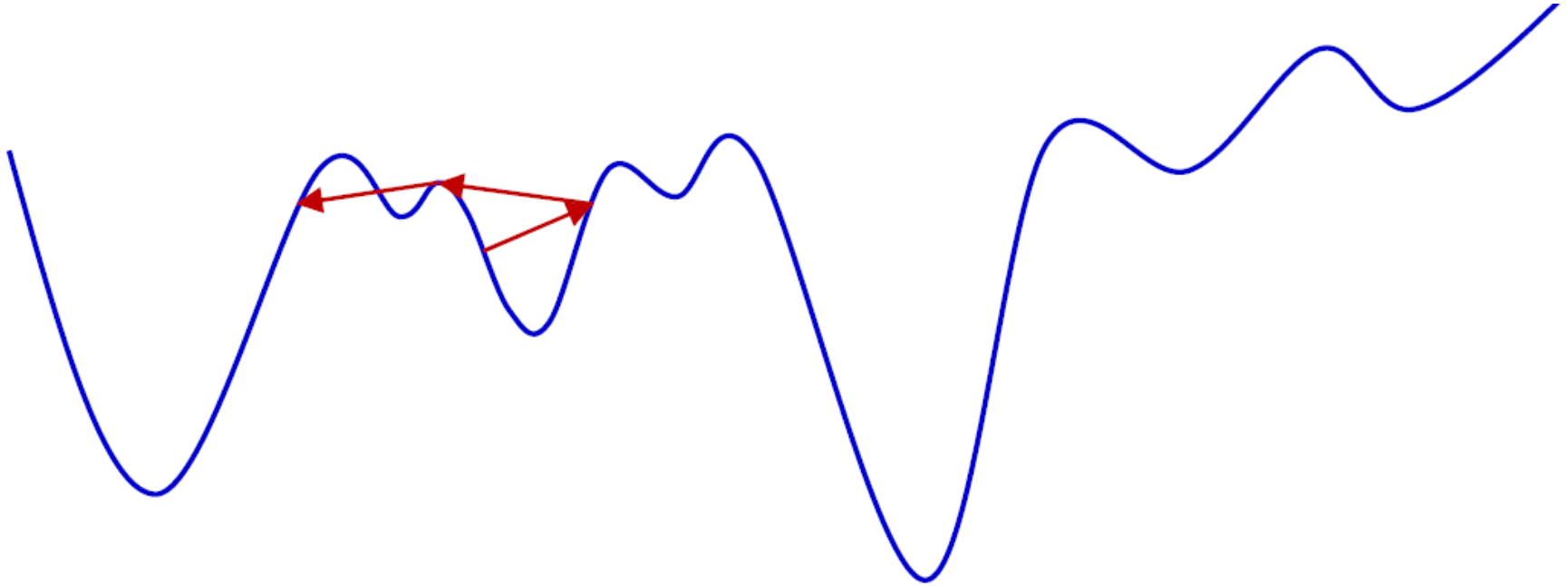
- many (different) local minima
- no (practical) way to say which is globally optimal



Convergence of Gradient Descent

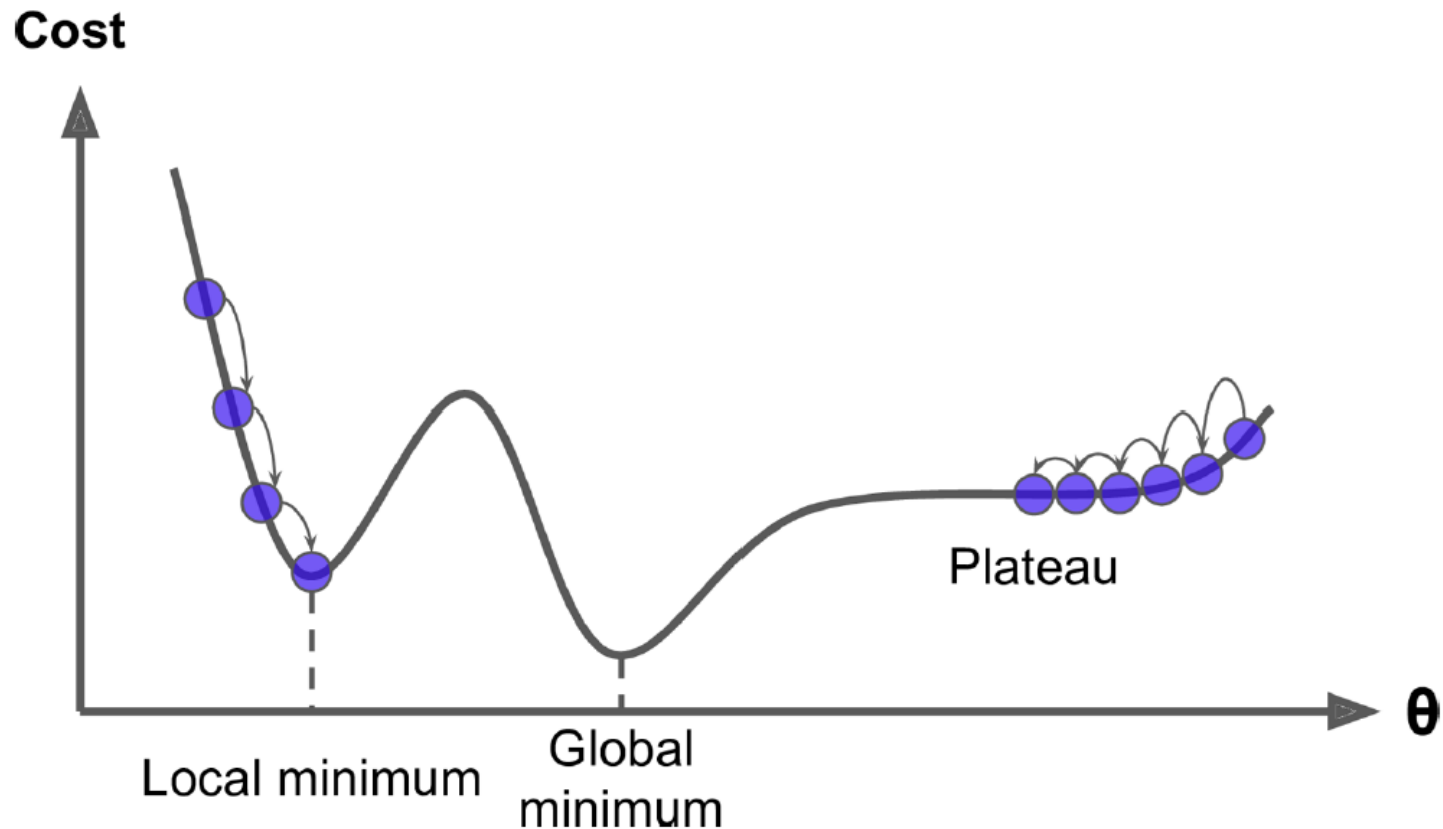


Convergence of Gradient Descent



- For complex models such as neural networks the loss function is often not convex

Convergence of Gradient Descent



Convergence of Gradient Descent

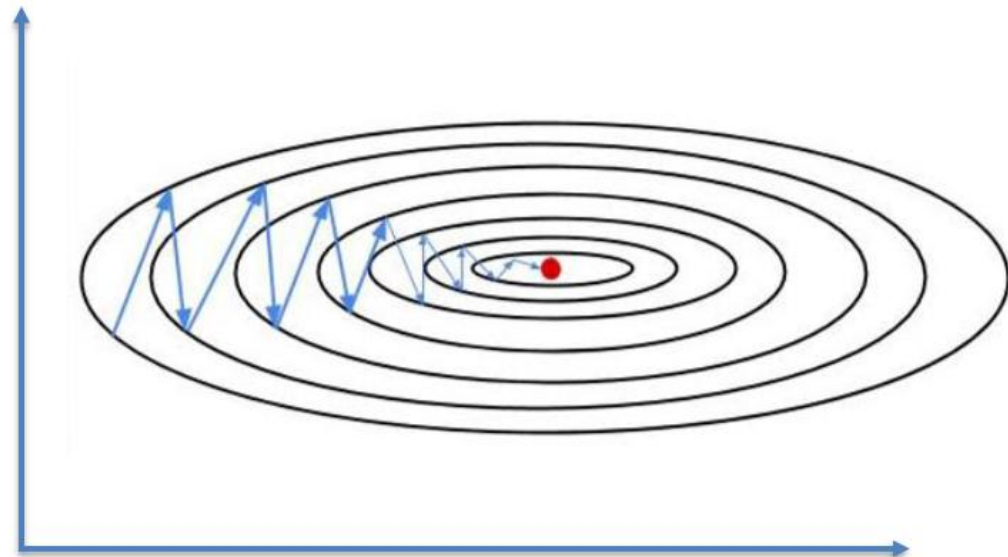
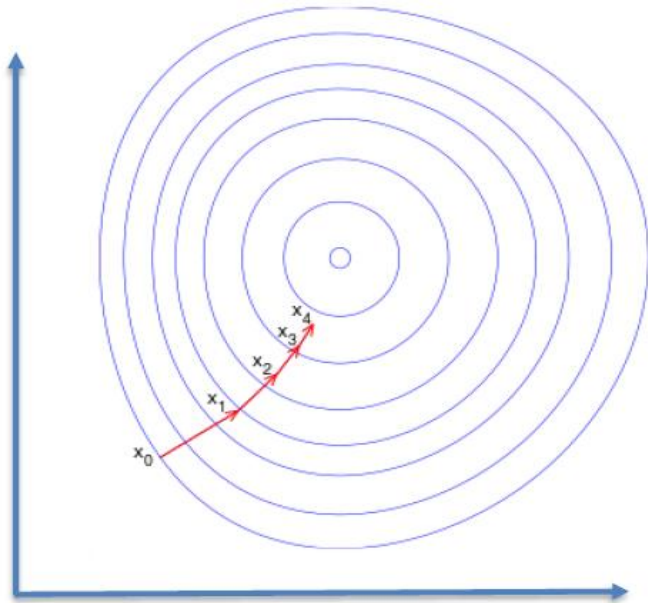
- Typical decay schedules

- Linear decay: $\eta_k = \frac{\eta_0}{k+1}$
- Quadratic decay: $\eta_k = \frac{\eta_0}{(k+1)^2}$
- Exponential decay: $\eta_k = \eta_0 e^{-\beta k}$, where $\beta > 0$

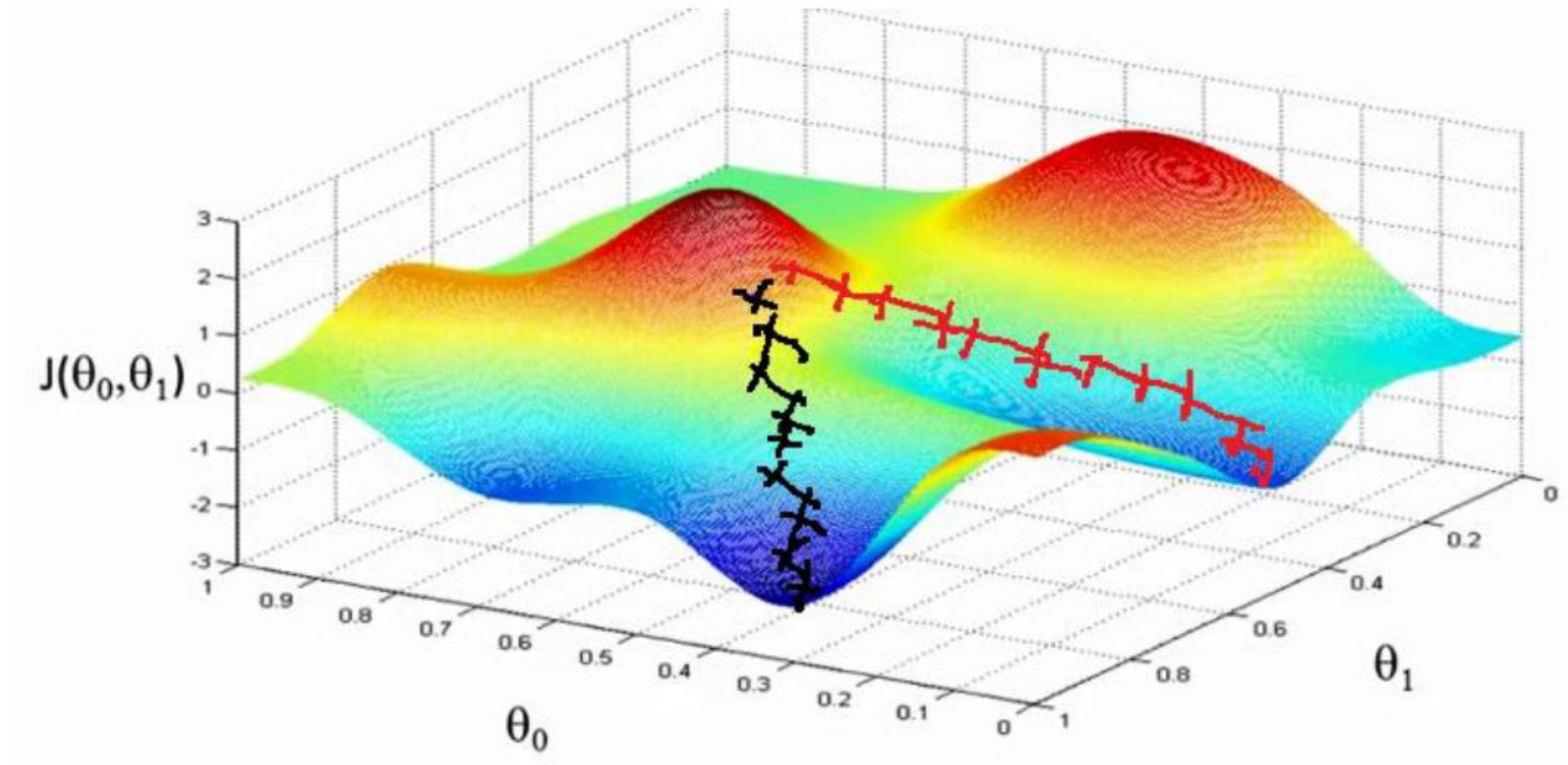
- A common approach (for nnets):

1. Train with a fixed learning rate η until loss (or performance on a held-out data set) stagnates
2. $\eta \leftarrow \alpha \eta$, where $\alpha < 1$ (typically 0.1)
3. Return to step 1 and continue training from where we left off

Gradient Descent: Multiple Dimensions



Gradient Descent: Multiple Dimensions

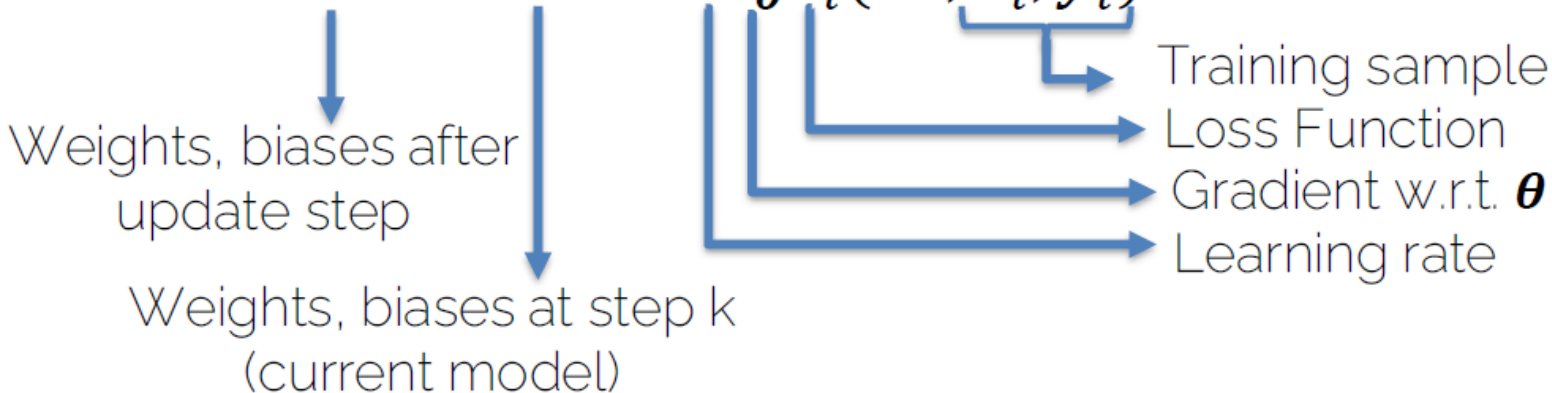


Gradient Descent: Single Data Point

- Given a loss function L and a single training sample $\{\mathbf{x}_i, \mathbf{y}_i\}$
- Find best model parameters $\boldsymbol{\theta} = \{\mathbf{W}, \mathbf{b}\}$
- Cost $L_i(\boldsymbol{\theta}, \mathbf{x}_i, \mathbf{y}_i)$
 - $\boldsymbol{\theta} = \arg \min L_i(\mathbf{x}_i, \mathbf{y}_i)$
- Gradient Descent:
 - Initialize $\boldsymbol{\theta}^1$ with 'random' values (more on that later)
 - $\boldsymbol{\theta}^{k+1} = \boldsymbol{\theta}^k - \alpha \nabla_{\boldsymbol{\theta}} L_i(\boldsymbol{\theta}^k, \mathbf{x}_i, \mathbf{y}_i)$
 - Iterate until convergence: $|\boldsymbol{\theta}^{k+1} - \boldsymbol{\theta}^k| < \epsilon$

Gradient Descent: Single Data Point

- $\boldsymbol{\theta}^{k+1} = \boldsymbol{\theta}^k - \alpha \nabla_{\boldsymbol{\theta}} L_i(\boldsymbol{\theta}^k, \mathbf{x}_i, \mathbf{y}_i)$



- $\nabla_{\boldsymbol{\theta}} L_i(\boldsymbol{\theta}^k, \mathbf{x}_i, \mathbf{y}_i)$ computed via backpropagation
- Typically: $\dim(\nabla_{\boldsymbol{\theta}} L_i(\boldsymbol{\theta}^k, \mathbf{x}_i, \mathbf{y}_i)) = \dim(\boldsymbol{\theta}) \gg 1 \text{ million}$

Gradient Descent: Multiple Data Points

- Update step for multiple samples

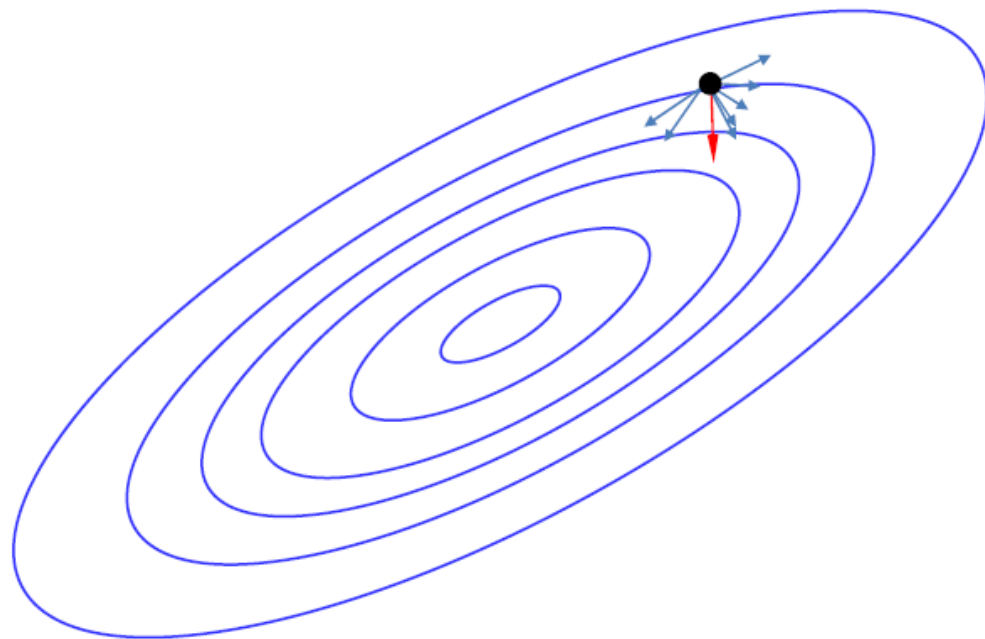
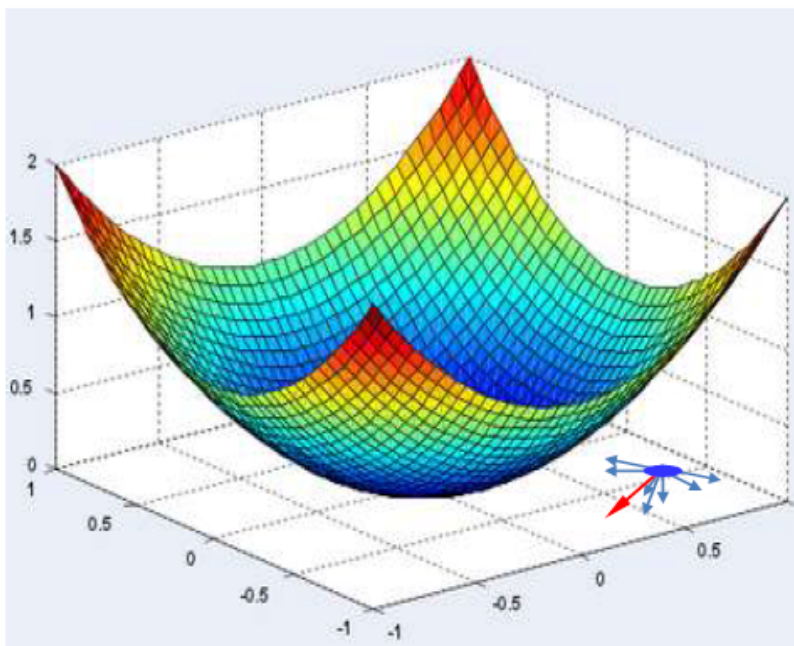
$$\boldsymbol{\theta}^{k+1} = \boldsymbol{\theta}^k - \alpha \nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}^k, \mathbf{x}_{\{1..n\}}, \mathbf{y}_{\{1..n\}})$$

- Gradient is average / sum over residuals

$$\nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}^k, \mathbf{x}_{\{1..n\}}, \mathbf{y}_{\{1..n\}}) = \frac{1}{n} \sum_{i=1}^n \underbrace{\nabla_{\boldsymbol{\theta}} L_i(\boldsymbol{\theta}^k, \mathbf{x}_i, \mathbf{y}_i)}$$

Reminder: this comes from backprop.

Gradient Descent: Multiple Data Points



- The individual training instances contribute different directions to the overall gradient
 - The final gradient points is the average of individual gradients
 - It points towards the *net* direction

Gradient Descent during Training

- Given large train set with n training samples $\{\mathbf{x}_i, \mathbf{y}_i\}$
 - Let's say 1 million labeled images
 - Let's say our network has 500k parameters
 - Gradient has 500k dimensions
 - $n = 1 \text{ million}$
- Extremely expensive to compute

Stochastic Gradient Descent

- If we have n training samples, we need to compute the gradient for all of them which is $O(n)$
- If we consider the problem as empirical risk minimization, we can express the total loss over the training data as the expectation of all the samples

$$\frac{1}{n} \left(\sum_{i=1}^n L_i(\boldsymbol{\theta}, \mathbf{x}_i, \mathbf{y}_i) \right) = \mathbb{E}_{i \sim [1, \dots, n]} [L_i(\boldsymbol{\theta}, \mathbf{x}_i, \mathbf{y}_i)]$$

Stochastic Gradient Descent

- The expectation can be approximated with a small subset of the data

$$\mathbb{E}_{i \sim [1, \dots, n]} [L_i(\boldsymbol{\theta}, \mathbf{x}_i, \mathbf{y}_i)] \approx \frac{1}{|S|} \sum_{j \in S} (L_j(\boldsymbol{\theta}, \mathbf{x}_j, \mathbf{y}_j)) \quad \text{with } S \subseteq \{1, \dots, n\}$$

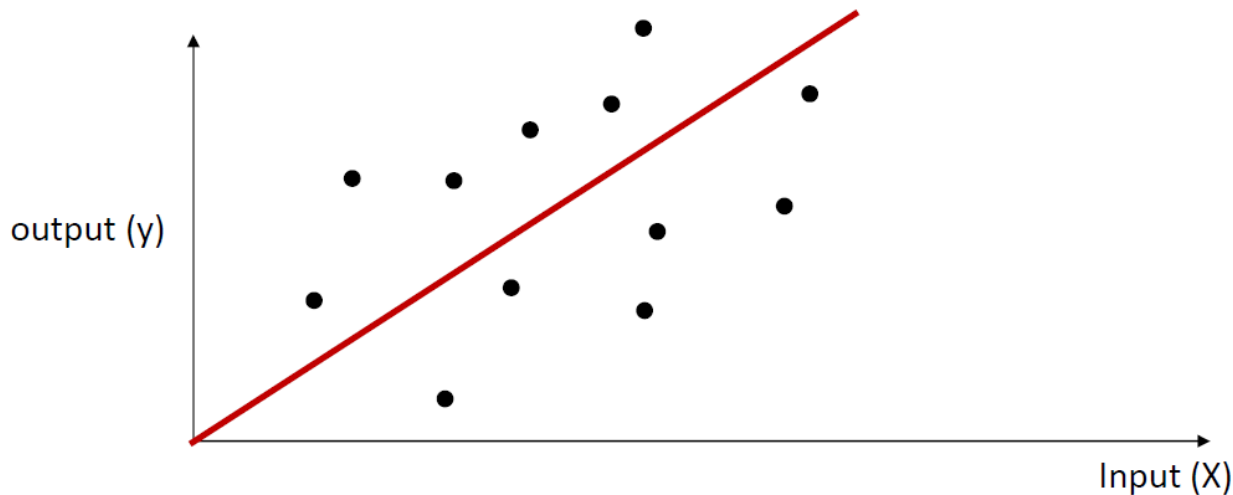
Minibatch

choose subset of trainset $m \ll n$

$$B_i = \{\{\mathbf{x}_1, \mathbf{y}_1\}, \{\mathbf{x}_2, \mathbf{y}_2\}, \dots, \{\mathbf{x}_m, \mathbf{y}_m\}\} \\ \{B_1, B_2, \dots, B_{n/m}\}$$

Stochastic Gradient Descent: Batch = 1

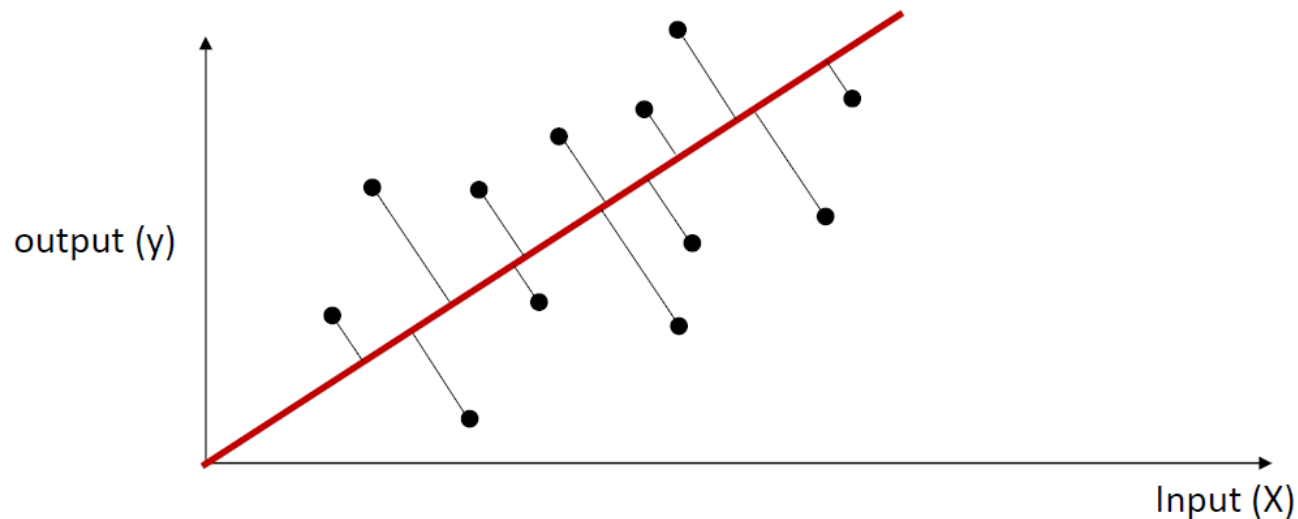
Incremental learning runs the risk of always chasing the latest input



- **Modelling problem:** Find a linear regression line (through origin) to model the data

Stochastic Gradient Descent: Batch = 1

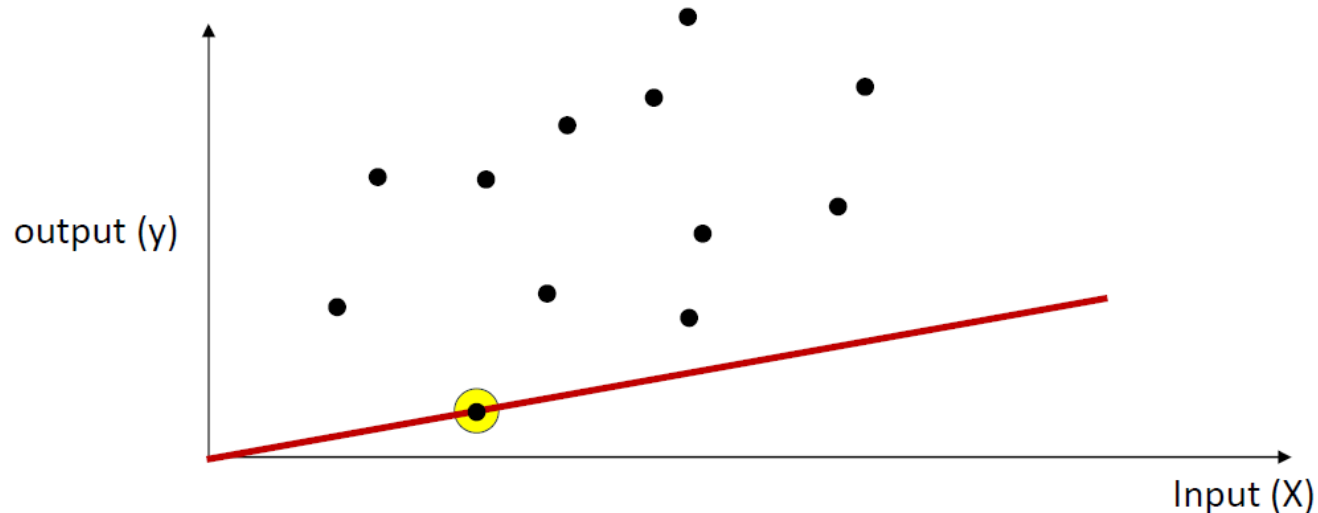
Incremental learning runs the risk of always chasing the latest input



- **Modelling problem:** Find a linear regression line (through origin) to model the data
 - **Batch processing:** Find the line through origin that has the lowest overall squared projection error w.r.t. data

Stochastic Gradient Descent: Batch = 1

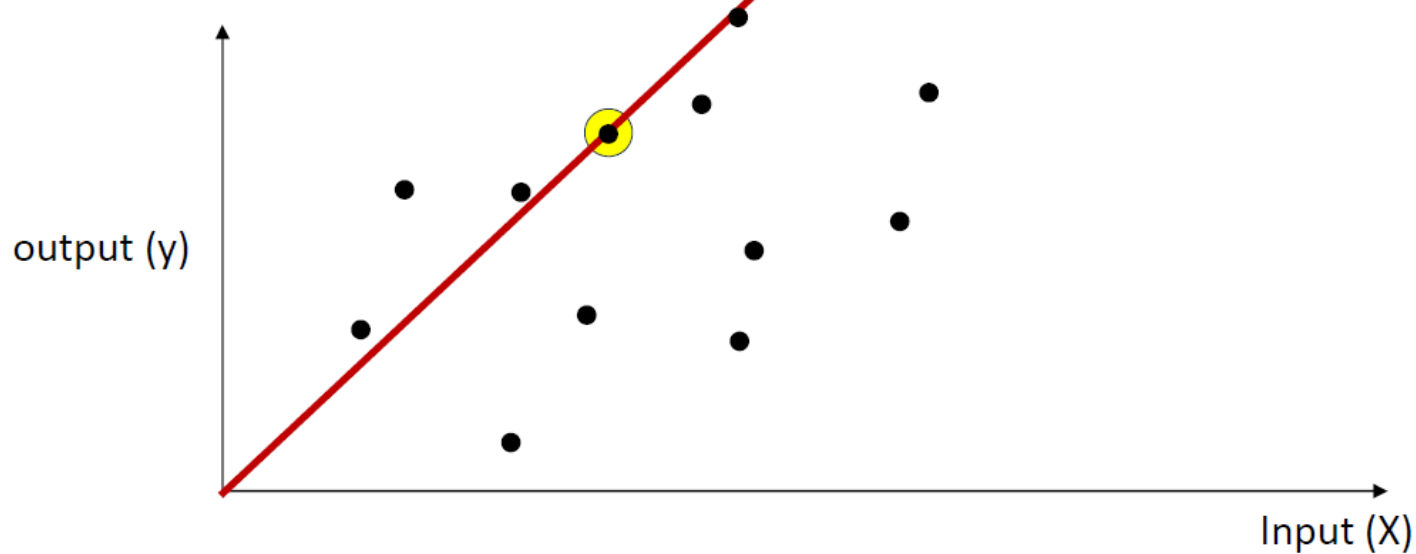
Incremental learning runs the risk of always chasing the latest input



- **Incremental learning:** Update the model to always minimize the error on the latest instance
 - It will never converge

Stochastic Gradient Descent: Batch = 1

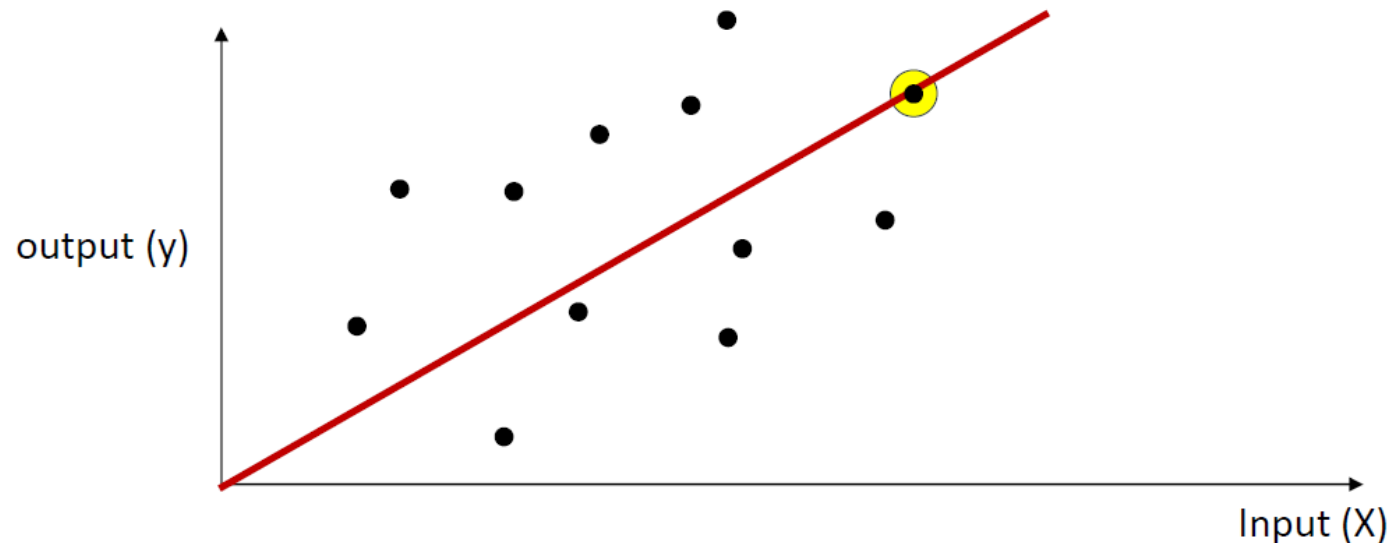
Incremental learning runs the risk of always chasing the latest input



- **Incremental learning:** Update the model to always minimize the error on the latest instance
 - It will never converge

Stochastic Gradient Descent: Batch = 1

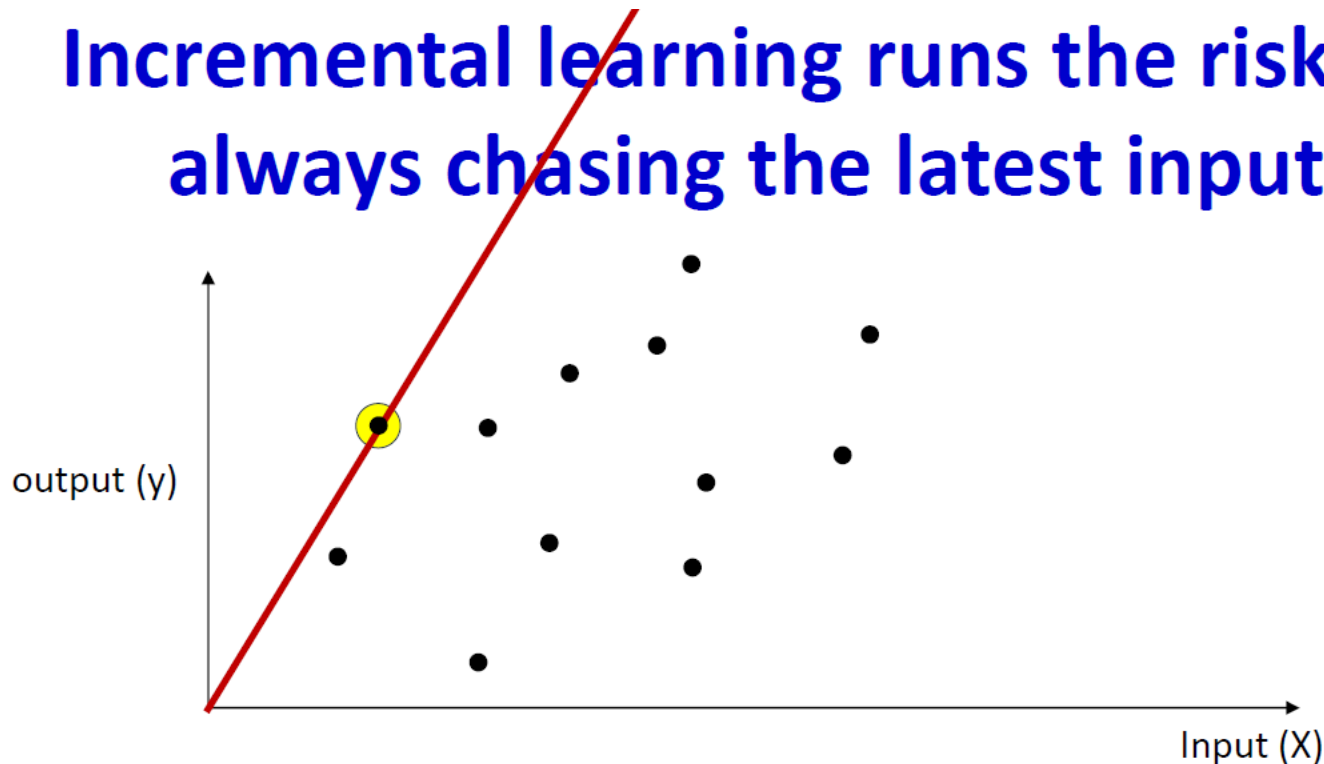
Incremental learning runs the risk of always chasing the latest input



- **Incremental learning:** Update the model to always minimize the error on the latest instance
 - It will never converge

Stochastic Gradient Descent: Batch = 1

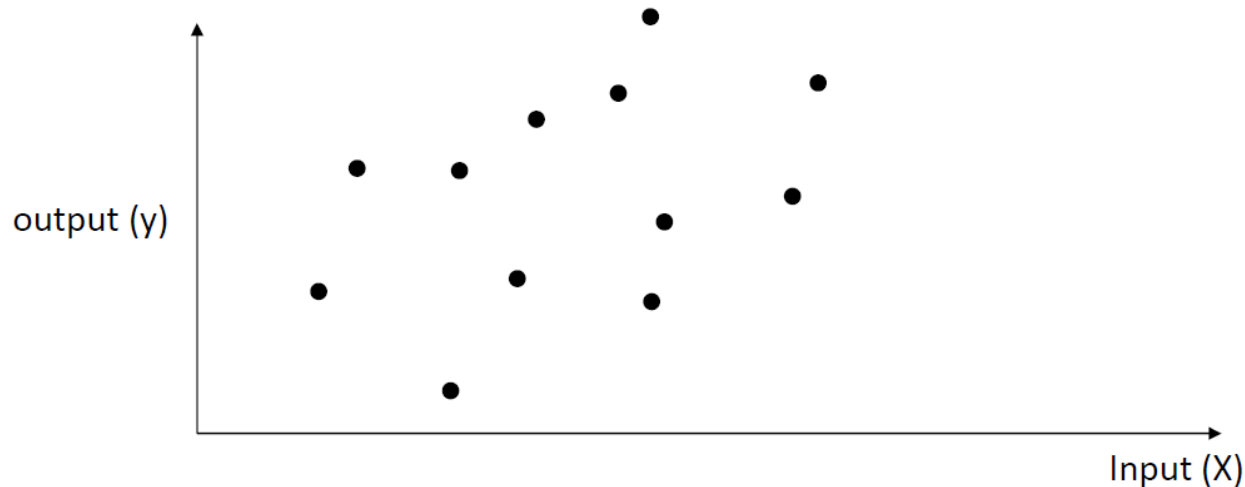
Incremental learning runs the risk of always chasing the latest input



- **Incremental learning:** Update the model to always minimize the error on the latest instance
 - It will never converge
 - **Solution?**

Stochastic Gradient Descent: Batch = 1

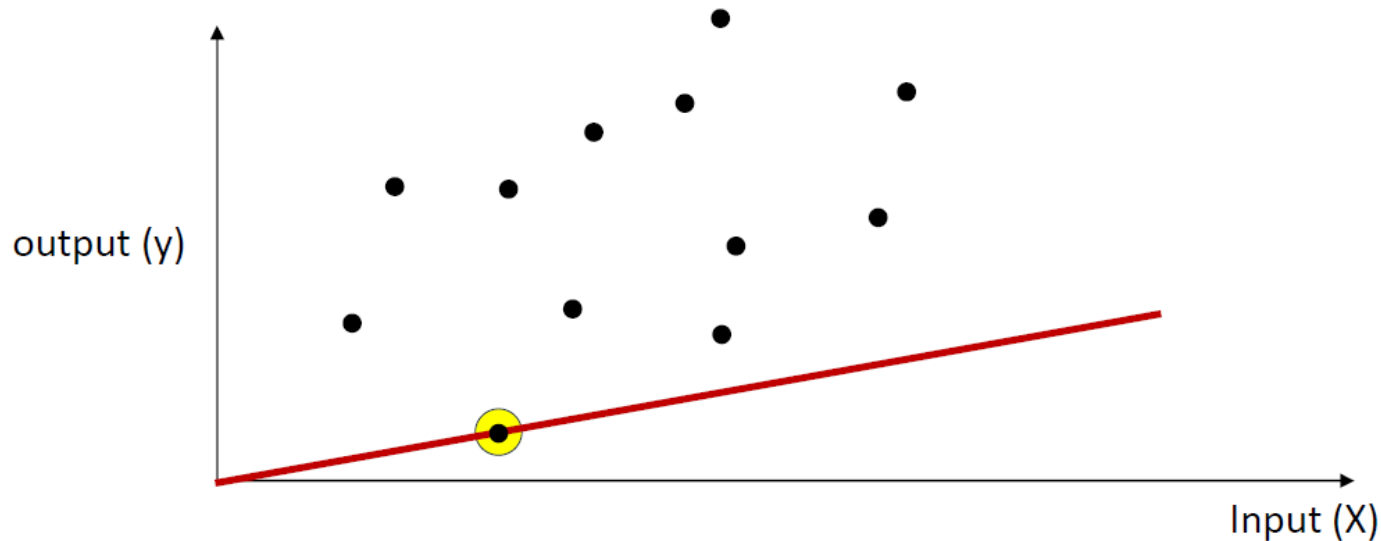
Incremental learning runs the risk of always chasing the latest input



- **Incremental learning:** Update the model to always minimize the error on the latest instance
 - Shrink the learning rate with iterations
 - With increasing iterations, it will swing less and less towards the new point

Stochastic Gradient Descent: Batch = 1

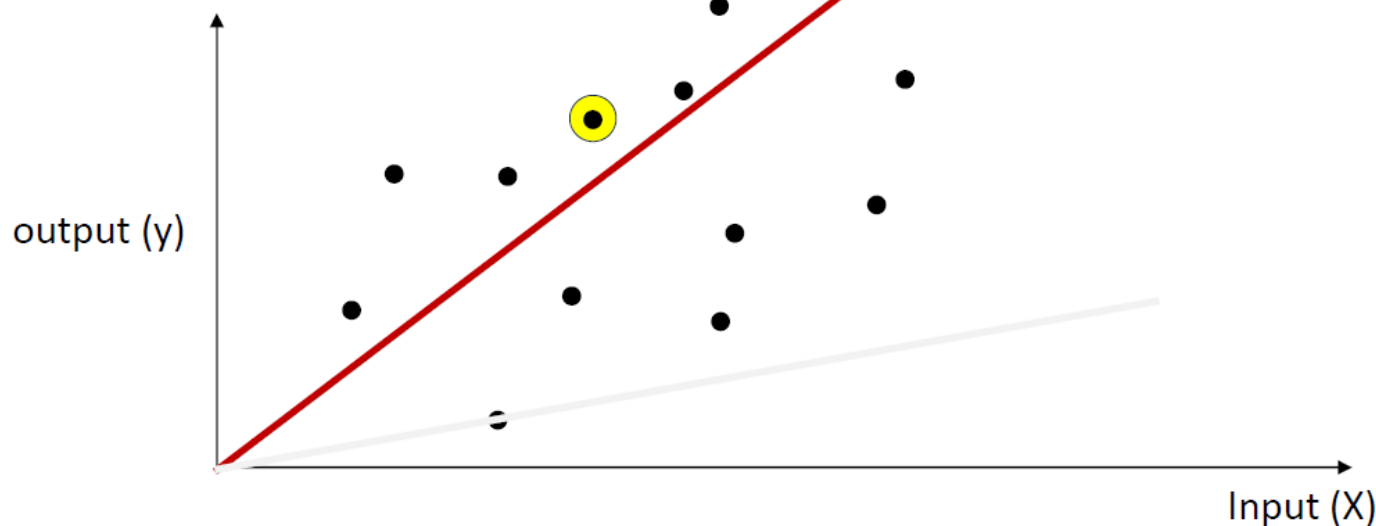
Incremental learning runs the risk of always chasing the latest input



- **Incremental learning:** Update the model to always minimize the error on the latest instance
 - Shrink the learning rate with iterations
 - With increasing iterations, it will swing less and less towards the new point

Stochastic Gradient Descent: Batch = 1

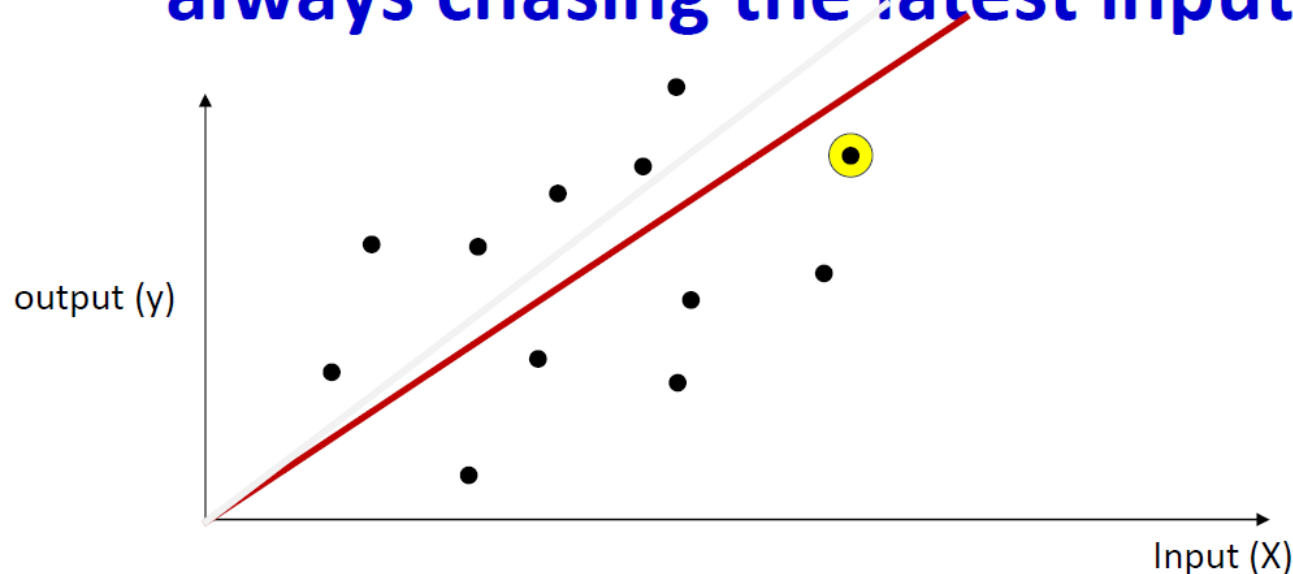
Incremental learning runs the risk of always chasing the latest input



- **Incremental learning:** Update the model to always minimize the error on the latest instance
 - Shrink the learning rate with iterations
 - With increasing iterations, it will swing less and less towards the new point

Stochastic Gradient Descent: Batch = 1

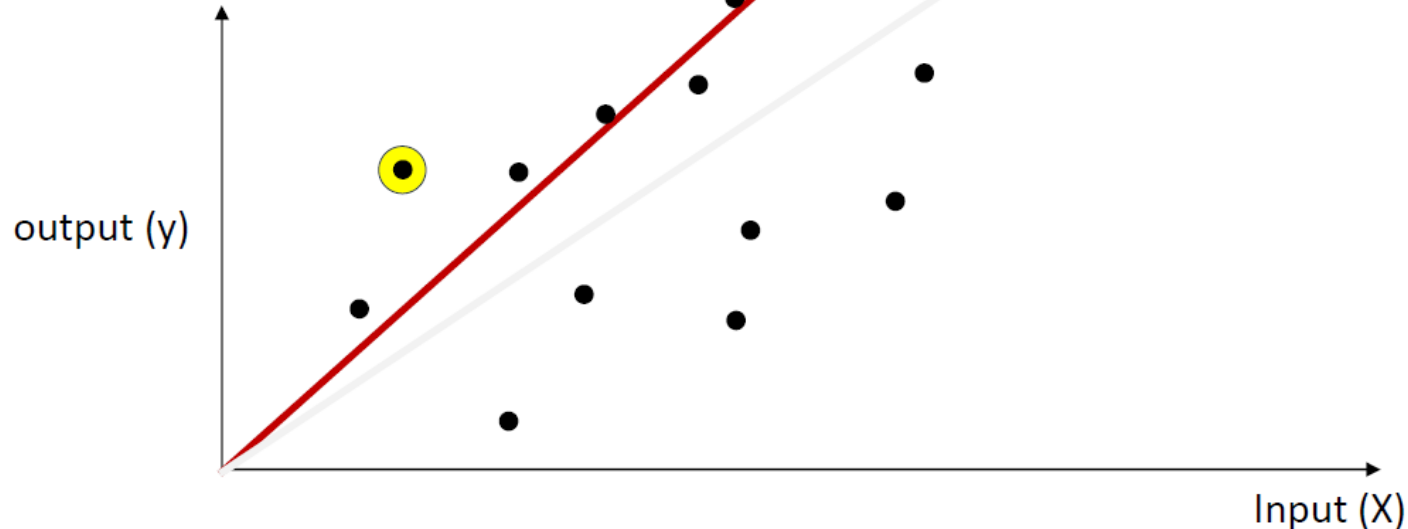
Incremental learning runs the risk of always chasing the latest input



- **Incremental learning:** Update the model to always minimize the error on the latest instance
 - Shrink the learning rate with iterations
 - With increasing iterations, it will swing less and less towards the new point

Stochastic Gradient Descent: Batch = 1

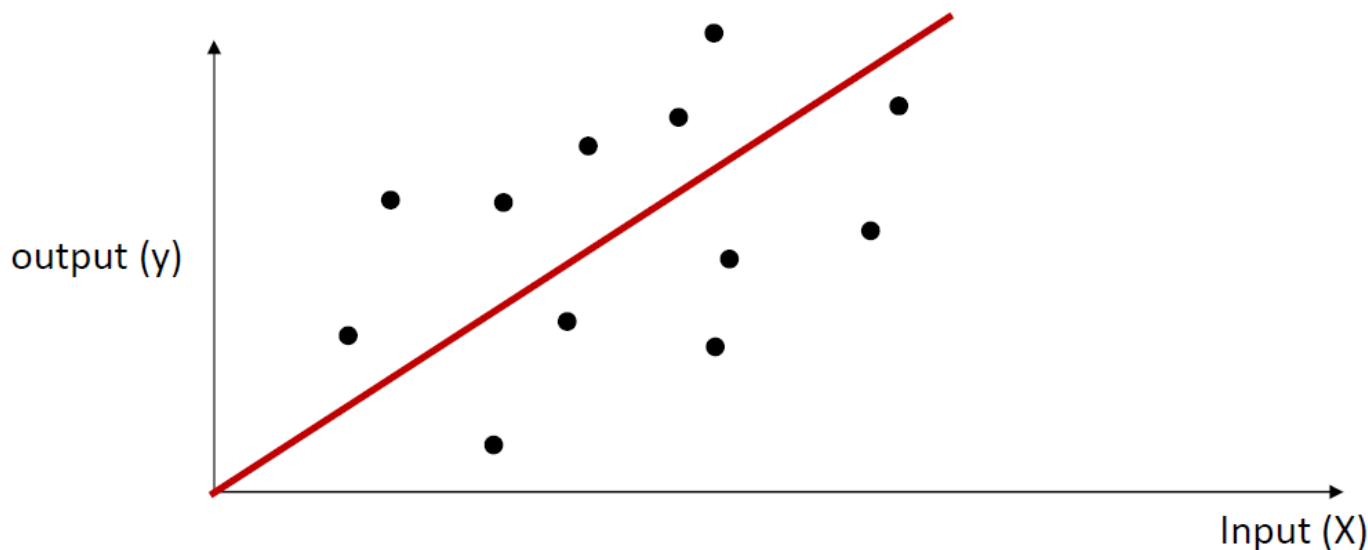
Incremental learning runs the risk of always chasing the latest input



- **Incremental learning:** Update the model to always minimize the error on the latest instance
 - Shrink the learning rate with iterations
 - With increasing iterations, it will swing less and less towards the new point

Stochastic Gradient Descent: Batch = 1

Incremental learning runs the risk of always chasing the latest input

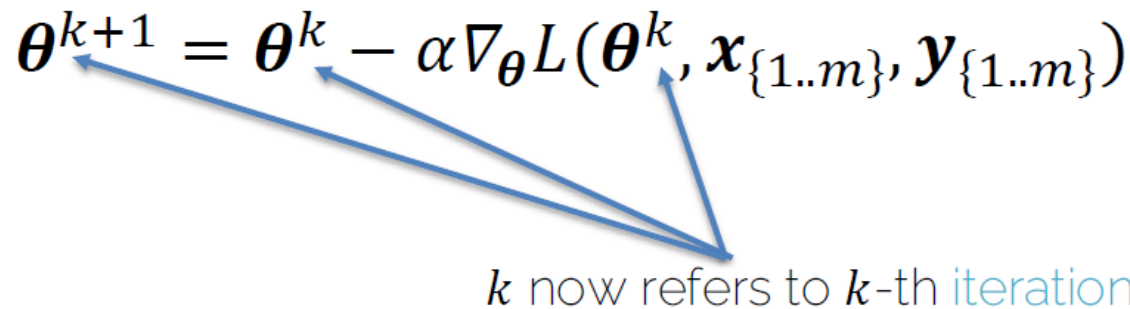


- **Incremental learning:** Update the model to always minimize the error on the latest instance
 - Shrink the learning rate with iterations
 - With increasing iterations, it will swing less and less towards the new point
 - Eventually arriving at the correct solution and not moving much from it further because the step sizes are now too small...

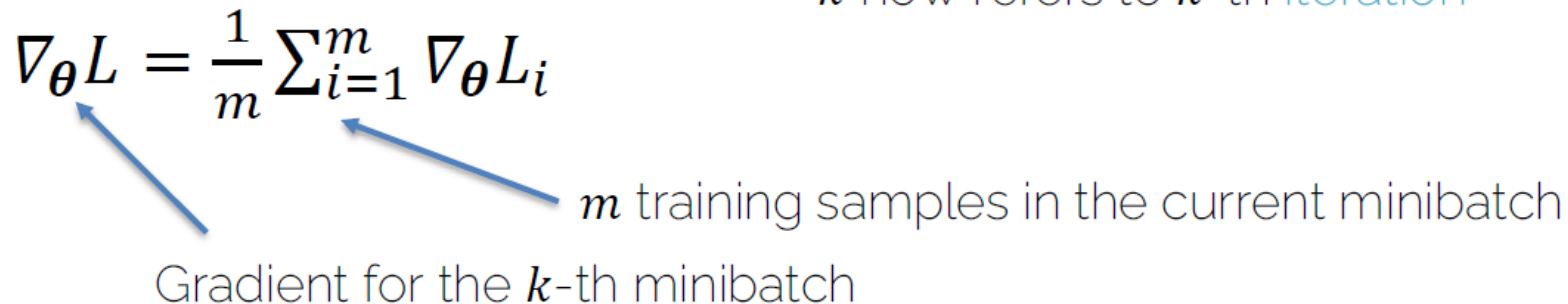
Stochastic Gradient Descent: Batch > 1

- Minibatch size is hyperparameter
 - Typically power of 2 $\rightarrow 8, 16, 32, 64, 128...$
 - Smaller batch size means greater variance in the gradients
 $\rightarrow$ noisy updates
 - Mostly limited by GPU memory (in backward pass)
 - E.g.,
 - Train set has $n = 2^{20}$ (about 1 million) images
 - With batch size $m = 64$: $B_1 \dots n/m = B_1 \dots 16,384$ minibatches
(Epoch = complete pass through training set)

Stochastic Gradient Descent: Batch > 1

$$\boldsymbol{\theta}^{k+1} = \boldsymbol{\theta}^k - \alpha \nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}^k, \mathbf{x}_{\{1..m\}}, \mathbf{y}_{\{1..m\}})$$


k now refers to k -th iteration

$$\nabla_{\boldsymbol{\theta}} L = \frac{1}{m} \sum_{i=1}^m \nabla_{\boldsymbol{\theta}} L_i$$


m training samples in the current minibatch

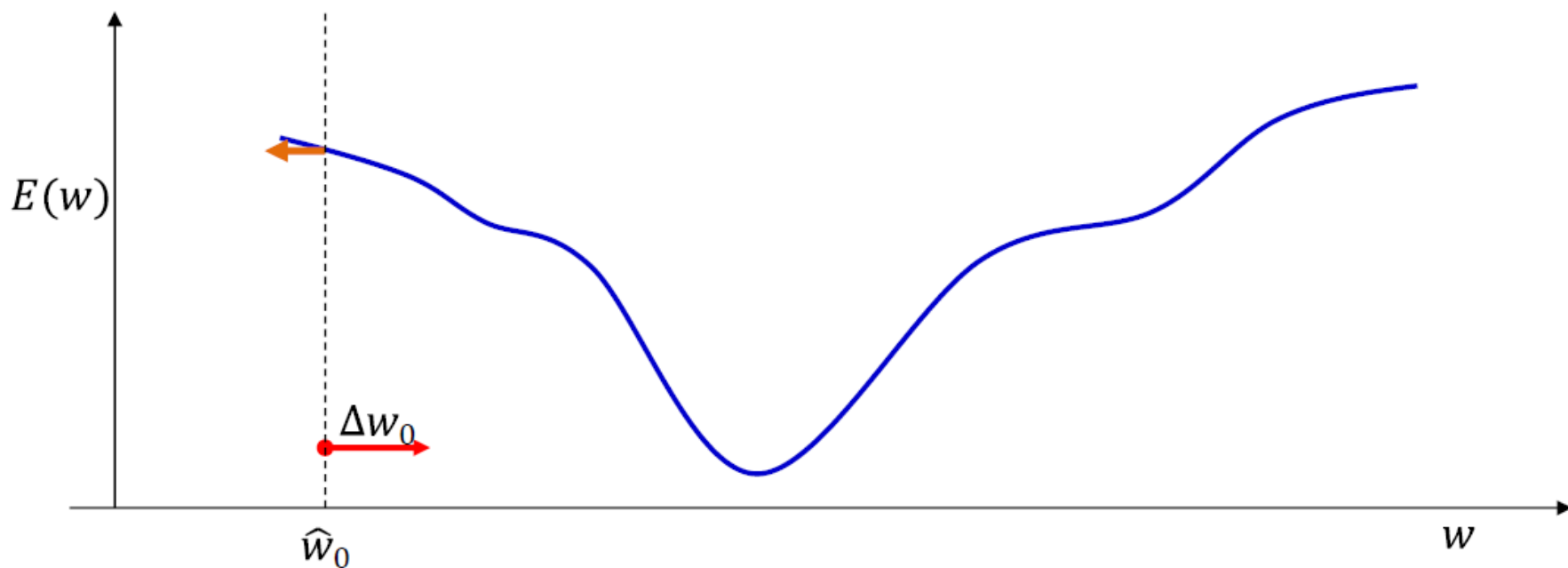
Gradient for the k -th minibatch

Note the terminology: iteration vs epoch

Resilient Propagation (Rprop)

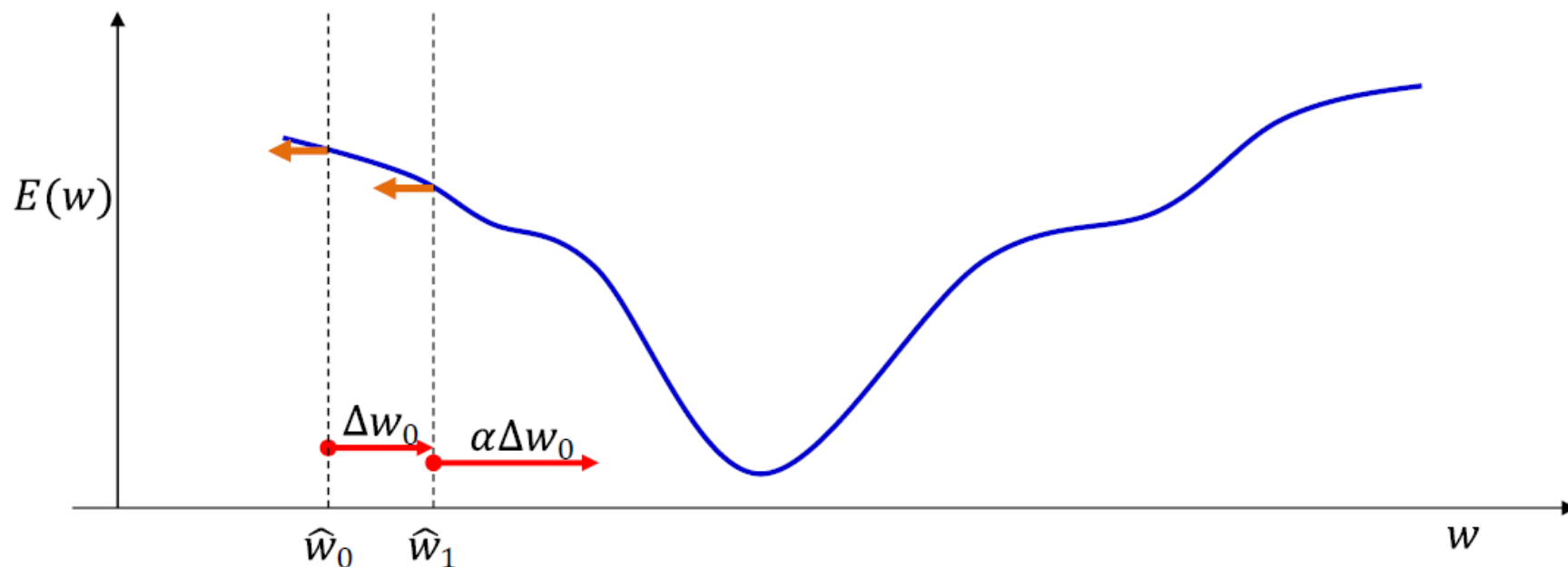
- At each time
 - If the derivative at the current location recommends continuing in the same direction as before (i.e. has not changed sign from earlier):
 - *increase* the step, and continue in the same direction
 - If the derivative has changed sign (i.e. we've overshoot a minimum)
 - *reduce* the step and reverse direction

Resilient Propagation (Rprop)



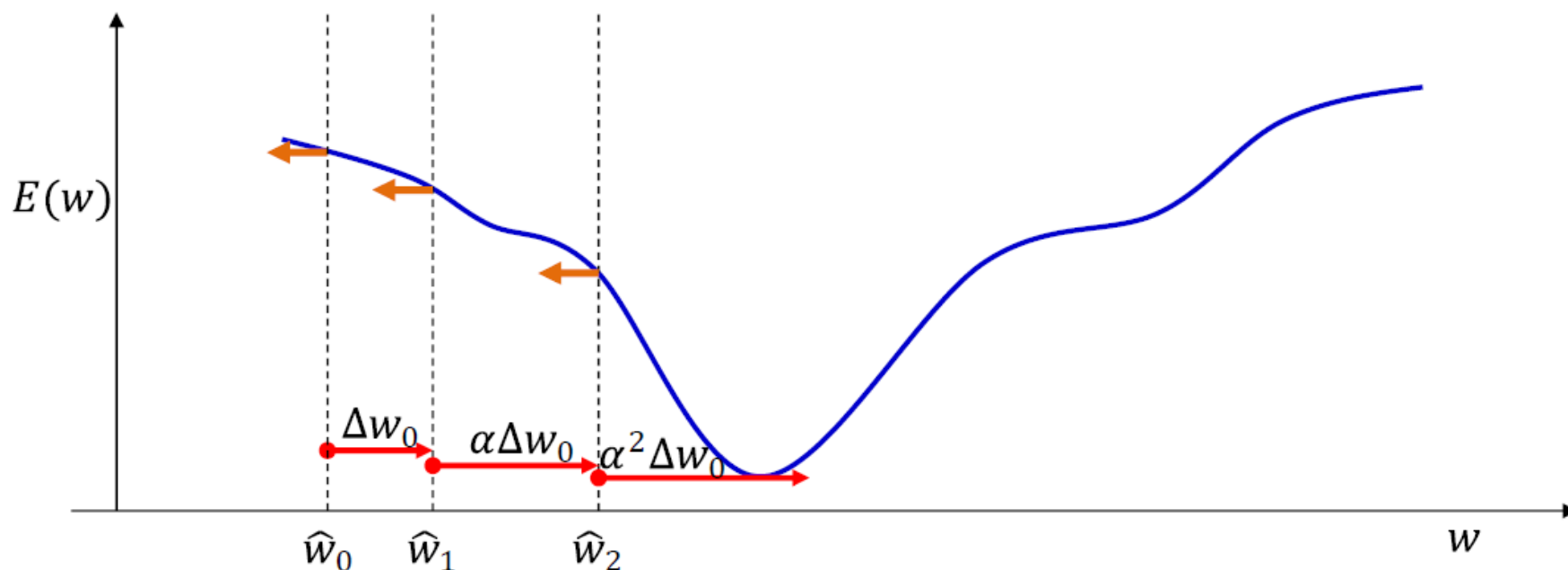
- Select an initial value $\hat{w}$ and compute the derivative
 - Take an initial step Δw against the derivative

Resilient Propagation (Rprop)



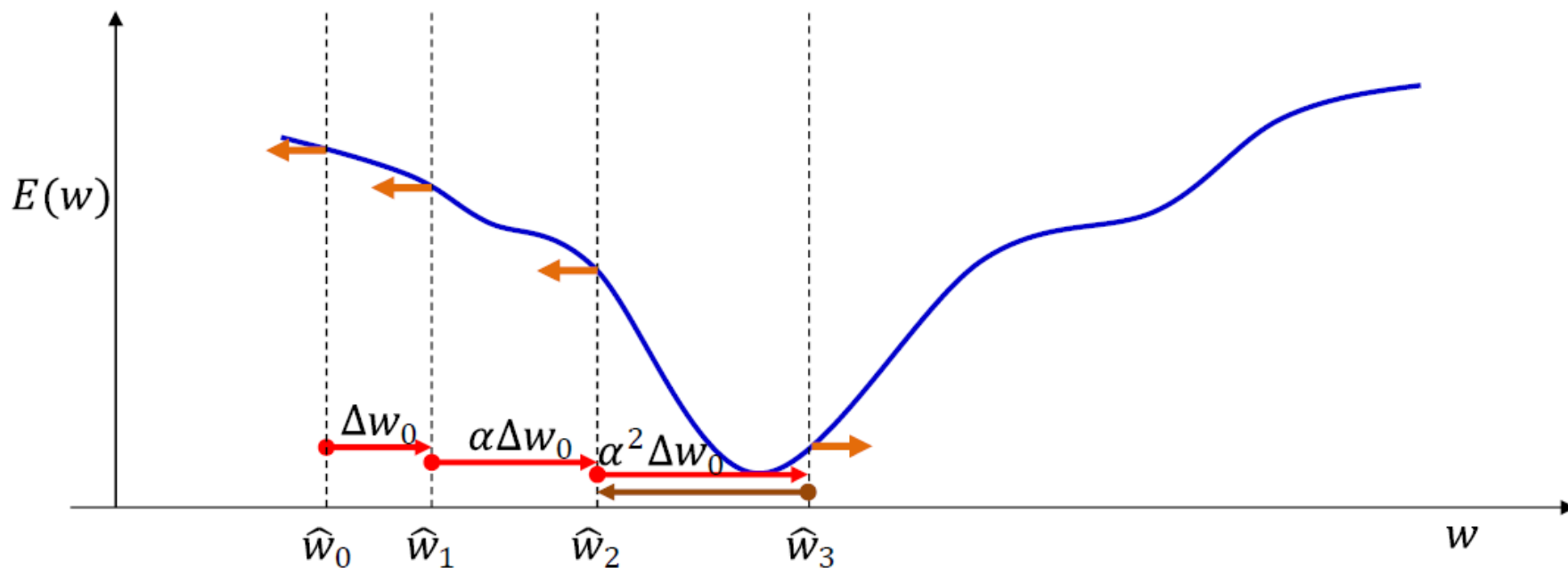
- Compute the derivative in the new location
 - If the derivative has not changed sign from the previous location, increase the step size and take a longer step

Resilient Propagation (Rprop)



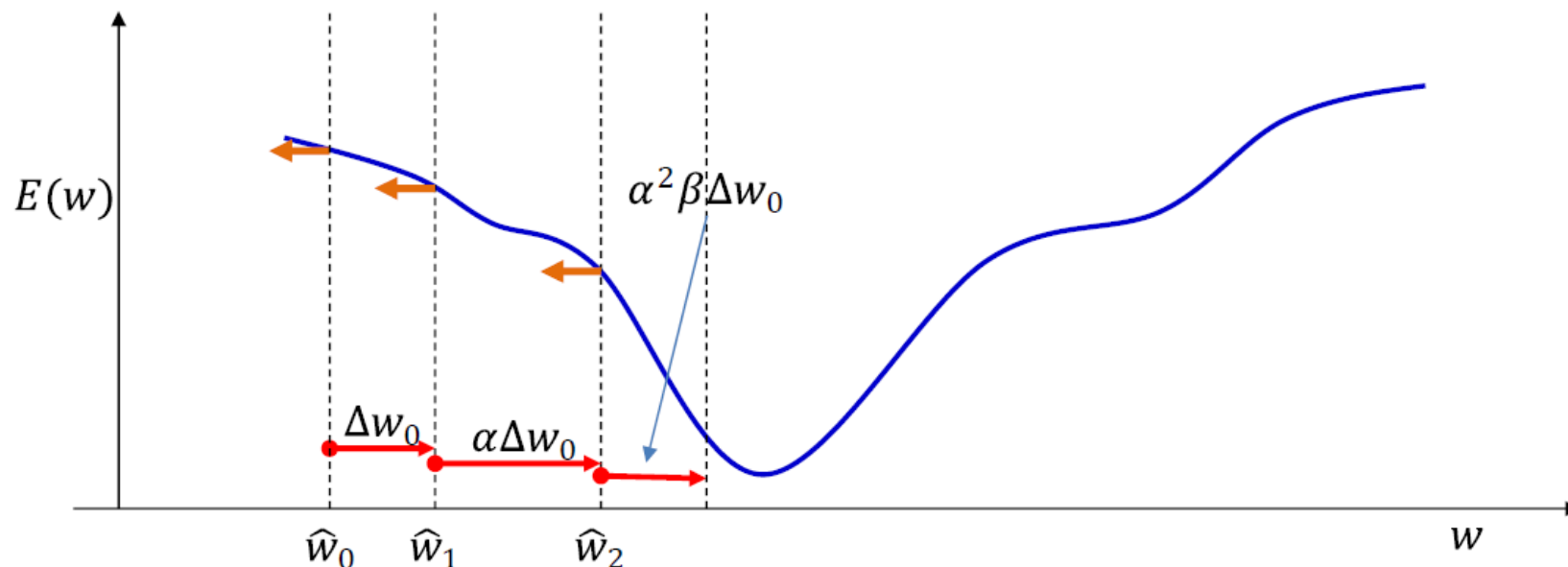
- Compute the derivative in the new location
 - If the derivative has not changed sign from the previous location, increase the step size and take a step

Resilient Propagation (Rprop)



- Compute the derivative in the new location
 - If the derivative has changed sign
 - Return to the previous location
 - $\hat{w} = \hat{w} + \Delta w$

Resilient Propagation (Rprop)



- Compute the derivative in the new location
 - If the derivative has changed sign
 - Return to the previous location
 - $\hat{w} = \hat{w} + \Delta w$
- Shrink the step
 - $\Delta w = \beta \Delta w$
- Take the smaller step forward
 - $\hat{w} = \hat{w} - \Delta w$

$\beta < 1$

Adaptive Gradient (Adagrad)

There were optimizers like Gradient Descent, Stochastic Gradient Descent, mini-batch SGD, all were used to reduce the loss function with respect to the weights. The value of η used to be the same for each weight.

But in Adagrad Optimizer the core idea is that **each weight has a different learning rate (η)**.

$$g_t = \frac{1}{m} \sum_{i \in B_t} \nabla_{\theta} \ell(\theta; x_i, y_i)$$

Problem?

$$G_t \rightarrow \infty \Rightarrow \text{LR} \rightarrow 0$$

$$G_t = G_{t-1} + g_t^2$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{G_t} + \epsilon} g_t$$

Large gradients shrink LR
Small gradients increase LR

RMS Prop

RMSprop uses a moving average of squared gradients to normalize the gradient. This normalization balances the step size, decreasing the step for large gradients to avoid exploding and increasing the step for small gradients to avoid vanishing.

$$E[g^2]_t = \rho E[g^2]_{t-1} + (1 - \rho)g_t^2$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{E[g^2]_t + \epsilon}} g_t$$

Momentum Based Method

Momentum adds inertia to gradient descent. It averages gradients over time, which stabilizes updates and helps move faster in shallow directions.

$$g_t = \frac{1}{m} \sum_{i \in B_t} \nabla_{\theta} \ell(\theta; x_i, y_i) \qquad \theta_{t+1} = \theta_t - \eta g_t$$

$$v_t = \beta v_{t-1} + (1 - \beta) g_t \qquad \beta \approx 0.9: \text{momentum coefficient}$$

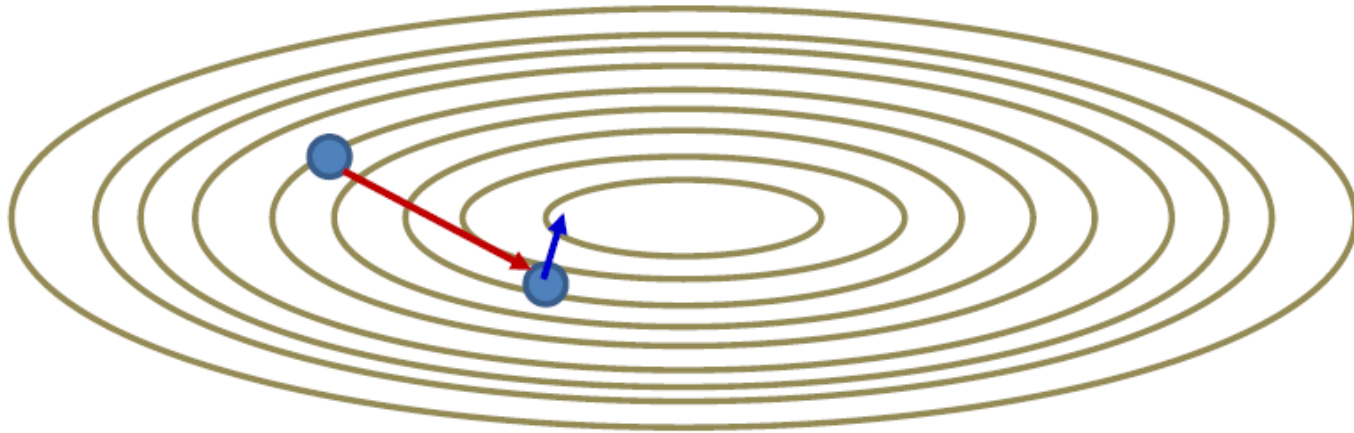
$$\theta_{t+1} = \theta_t - \eta v_t$$

Accumulates past gradients → smoother updates

Reduces zig-zag behavior

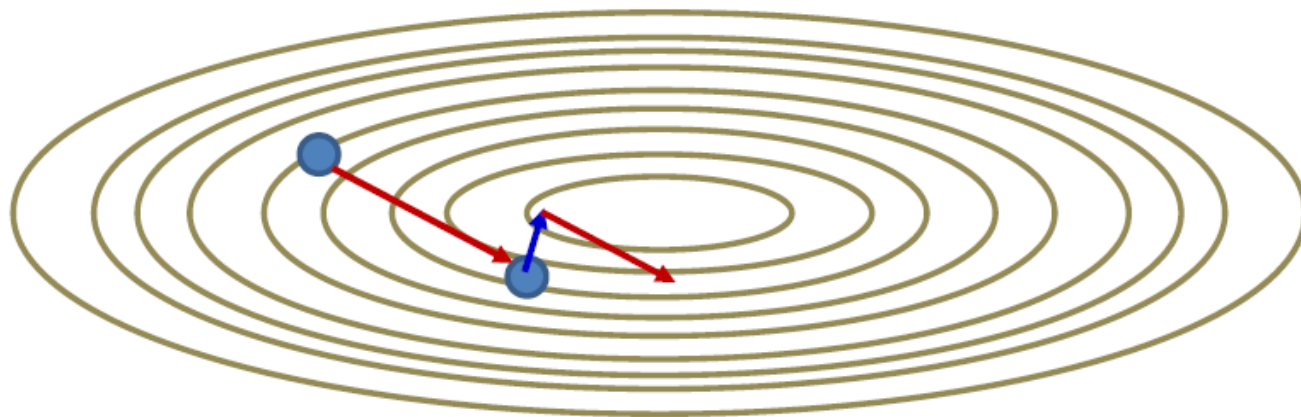
Accelerates along consistent directions

The Momentum Method



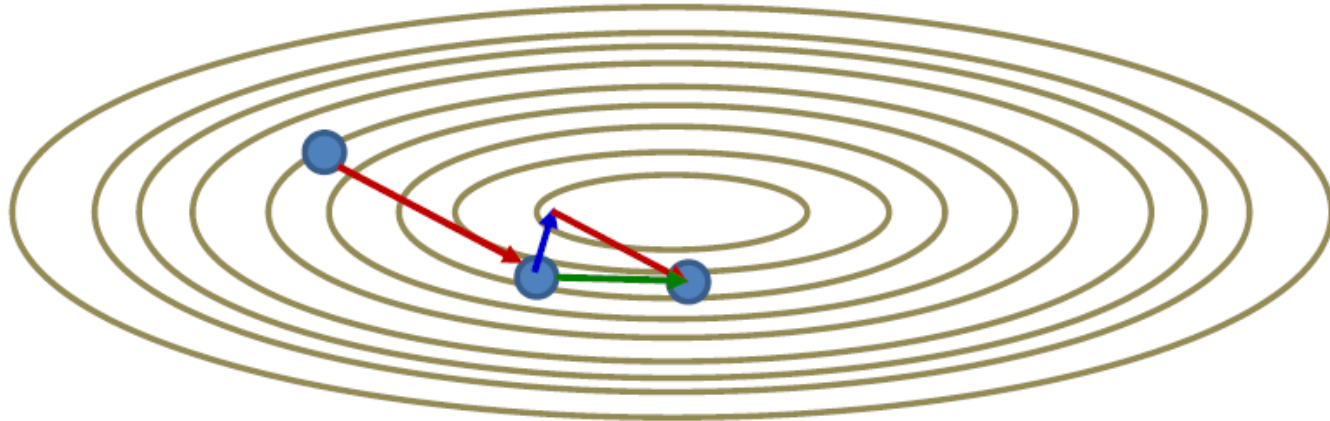
- At any iteration, to compute the current step:
 - First computes the gradient step at the current location

The Momentum Method



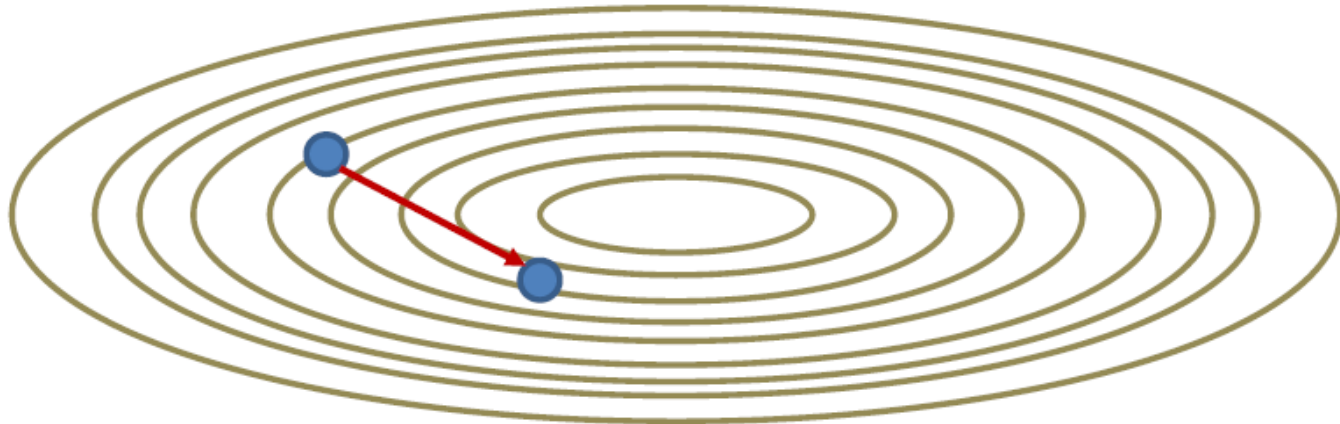
- At any iteration, to compute the current step:
 - First computes the gradient step at the current location
 - Then adds in the scaled *previous* step
 - Which is actually a running average

The Momentum Method



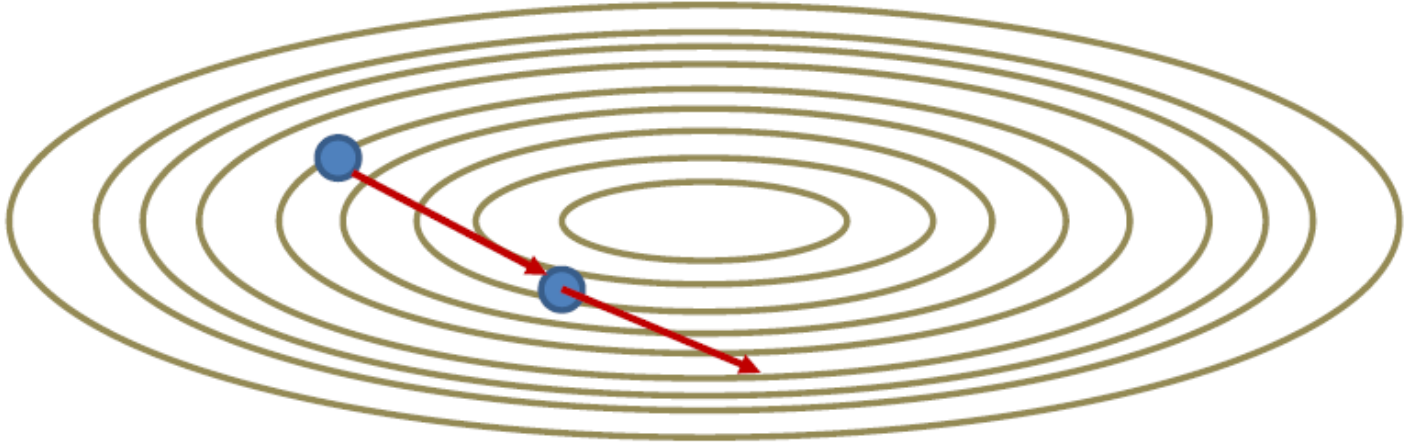
- At any iteration, to compute the current step:
 - First computes the gradient step at the current location
 - Then adds in the scaled *previous* step
 - Which is actually a running average
 - To get the final step

Nestorov's Accelerated Gradient



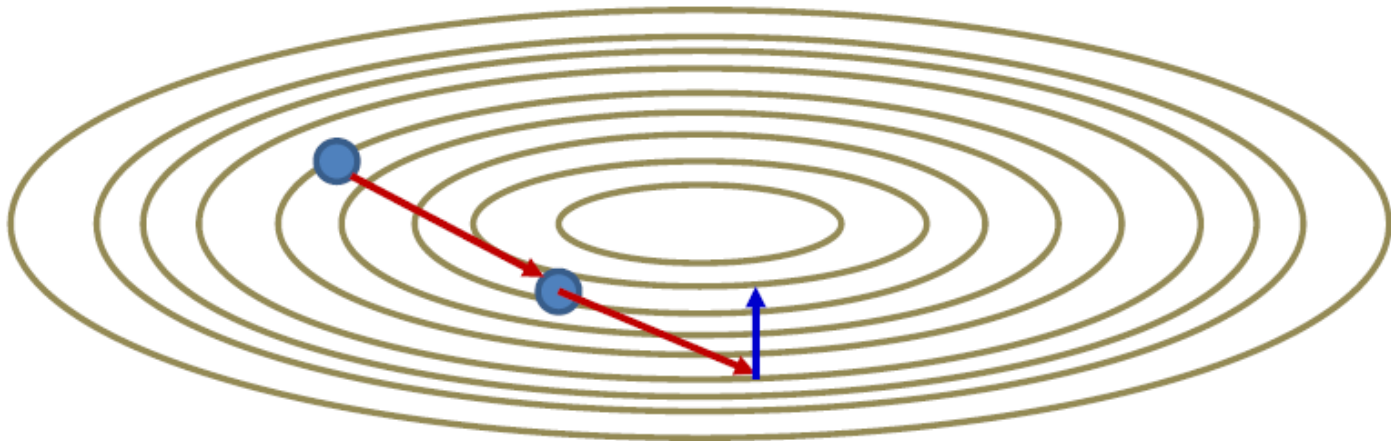
- Change the order of operations
- At any iteration, to compute the current step:

Nestorov's Accelerated Gradient



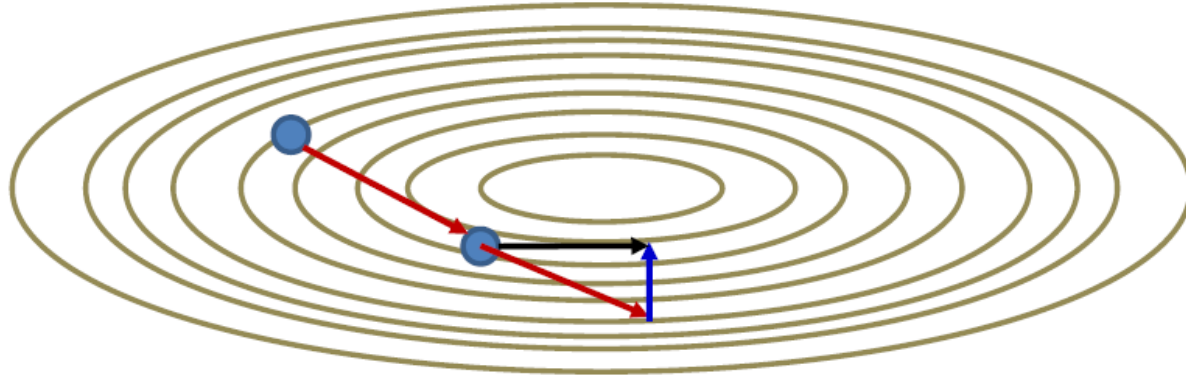
- Change the order of operations
- At any iteration, to compute the current step:
 - First extend the previous step

Nestorov's Accelerated Gradient



- Change the order of operations
- At any iteration, to compute the current step:
 - First extend the previous step
 - Then compute the gradient step at the resultant position

Nestorov's Accelerated Gradient



- Change the order of operations
- At any iteration, to compute the current step:
 - First extend the previous step
 - Then compute the gradient step at the resultant position
 - Add the two to obtain the final step

ADAM Optimizer

Adam combines the two approaches (momentum and variable learning rate). We can see how the θ update looks similar to the RMSProp update, amplified with Momentum, so we get the best of both worlds.

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$g_t = \frac{1}{m} \sum_{i \in B_t} \nabla_{\theta} \ell(\theta; x_i, y_i)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

$$\theta_{t+1} = \theta_t - \eta g_t$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

Thank you